

Карагандинский национальный исследовательский
университет имени академика Е.А. Букетова
Факультет математики и информационных технологий
Кафедра прикладной математики и информатики

ОТЧЁТ

по учебной практике

за 1 день (25.05.2026)

Образовательная программа: 6В06103 — Информационные системы

Выполнил: студент гр. ИС-25-ЗР
Шакир Нурасыл

Тема 1. Программирование целочисленных арифметических операций

Теоретические вопросы

1. Что такое целочисленный тип данных?

Целочисленный тип данных — это тип переменной, который хранит только целые числа (без дробной части). В C++ к целочисленным типам относятся: `int`, `short`, `long`, `long long`, а также их беззнаковые варианты (`unsigned int` и т.д.).

2. Какие арифметические операции используются в программировании?

В программировании используются следующие арифметические операции:

+ (сложение), - (вычитание), * (умножение), / (деление), % (остаток от деления).

3. Чем отличается целочисленное деление от обычного?

При целочисленном делении дробная часть отбрасывается (не округляется). Например: $15 / 4 = 3$, а не 3.75. При обычном (вещественном) делении результат сохраняет дробную часть, если хотя бы один операнд типа `float` или `double`.

4. Для чего используется оператор остатка от деления?

Оператор % возвращает остаток от целочисленного деления. Например: $15 \% 4 = 3$. Применяется для проверки чётности числа ($n \% 2 == 0$), для извлечения цифр числа ($n \% 10$ — последняя цифра), а также в задачах на циклическое повторение.

Практическое задание

Задание 1. Простые арифметические операции

Написать программу, которая вводит два целых числа и выводит сумму, разность, произведение, результат целочисленного деления и остаток от деления.

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;
    cout << "Введите два числа: ";
    cin >> a >> b;
    cout << "Сумма = " << a + b << endl;
    cout << "Разность = " << a - b << endl;
    cout << "Произведение = " << a * b << endl;
    cout << "Деление = " << a / b << endl;
    cout << "Остаток = " << a % b << endl;
    return 0;
}
```

Результат выполнения программы:

```
Введите два числа: 15 4
Сумма = 19
Разность = 11
Произведение = 60
Деление = 3
Остаток = 3
```

Задание 2. Работа с цифрами числа

Дано трёхзначное число. Найти сумму цифр, произведение цифр и вывести число в обратном порядке.

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    cout << "Введите трёхзначное число: ";
    cin >> n;
    int c1 = n / 100; // цифра сотен
    int c2 = (n / 10) % 10; // цифра десятков
    int c3 = n % 10; // цифра единиц
    cout << "Сумма цифр = " << c1 + c2 + c3 << endl;
    cout << "Произведение цифр = " << c1 * c2 * c3 << endl;
}
```

```
cout << "Обратное число = " << c3*100 + c2*10 + c1 << endl;  
return 0;  
}
```

Результат выполнения программы:

```
Введите трёхзначное число: 325  
Сумма цифр = 10  
Произведение цифр = 30  
Обратное число = 523
```

Задание 3. Практическая задача

Написать программу вычисления количества минут в N часах и количества секунд в N минутах.

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int n;  
    cout << "Введите N: ";  
    cin >> n;  
    cout << "Минут в " << n << " часах: " << n * 60 << endl;  
    cout << "Секунд в " << n * 60 << " минутах: " << n * 3600 << endl;  
    return 0;  
}
```

Результат выполнения программы:

```
Введите N: 3  
Минут в 3 часах: 180  
Секунд в 180 минутах: 10800
```

Контрольные вопросы

1. Что такое переменная?

Переменная — это именованная область памяти, которая хранит значение определённого типа. Значение переменной можно изменять во время выполнения программы.

2. Какие этапы включает разработка программы?

Постановка задачи → Составление алгоритма → Написание кода → Компиляция → Тестирование и отладка.

3. Что такое оператор присваивания?

Оператор присваивания (=) записывает значение правой части выражения в переменную левой части. Например: $a = 5$ — в переменную a записывается значение 5.

4. Как работает операция %?

Операция % возвращает остаток от целочисленного деления левого операнда на правый. Например: $17 \% 5 = 2$, так как $17 = 5 * 3 + 2$. Знак результата совпадает со знаком левого операнда.

Тема 2. Изучение синтаксиса и семантики языка. Работа с алфавитом языка

Теоретические вопросы

1. Что такое синтаксис языка программирования?

Синтаксис — это набор правил, которые определяют, как должен быть записан код программы. Он описывает допустимые конструкции языка: как расставлять скобки, точки с запятой, ключевые слова и т.д.

2. Что такое семантика?

Семантика — это смысловое значение конструкций языка, то есть что именно они выполняют. Программа может быть синтаксически правильной, но семантически ошибочной (например, деление на ноль — синтаксически верно, но по смыслу ошибка).

3. Какие символы входят в алфавит языка?

Алфавит C++ включает: буквы латинского алфавита (a–z, A–Z) и знак подчёркивания (_); цифры (0–9); специальные символы (+, -, *, /, =, <, >, :, ::, {, }, (,), [,] и др.); пробельные символы (пробел, табуляция, перевод строки).

4. Что такое идентификатор?

Идентификатор — это имя, которое программист даёт переменной, функции, классу и т.д. Правила: начинается с буквы или _, содержит буквы/цифры/_, не совпадает с ключевым словом, чувствителен к регистру.

5. Какие существуют ключевые слова?

Ключевые слова — зарезервированные слова языка, которые нельзя использовать как идентификаторы. Примеры: int, float, double, char, bool, void, if, else, for, while, do, return, class, struct, true, false, const, new, delete.

Практическое задание

Задание 1. Определение ошибок

Найти и исправить ошибки в программе.

Исходный код с ошибками:

```
include <iostream>

using namespace std

int main()
{
    int lnumber = 10;
    cout << Number;
}
```

Список ошибок и их причины:

- 1) `include <iostream>` — нет символа `#`. Директивы препроцессора обязательно начинаются с `#`, правильно: `#include <iostream>`
- 2) `using namespace std` — нет точки с запятой. Каждая инструкция в C++ должна заканчиваться `;`
- 3) `int 1number` — идентификатор начинается с цифры. Имена переменных не могут начинаться с цифры, правильно: `int number`
- 4) `cout << Number` — переменная называется `1number`, а не `Number`. C++ чувствителен к регистру, плюс исходное имя было некорректным
- 5) Нет `return 0` — функция `main()` обязана возвращать целое число

Исправленный вариант:

```
#include <iostream>

using namespace std;

int main()
{
    int number = 10;
    cout << number;
    return 0;
}
```

Результат выполнения программы:

```
10
```

Задание 2. Работа с идентификаторами

Создать программу с 5 корректными и 5 некорректными идентификаторами.

```
#include <iostream>
using namespace std;

int main()
{
    // 5 корректных идентификаторов:
    int myVar    = 1;    // начинается с буквы — правильно
    int _count   = 2;    // начинается с _ — правильно
    int number1  = 3;    // цифра в конце — правильно
    int totalSum = 4;    // camelCase — правильно
    int MAX_VALUE = 5;    // заглавные буквы и _ — правильно

    // 5 некорректных идентификаторов (закомментированы):
    // int lvar    = 6;    // начинается с цифры — ошибка
    // int my var  = 7;    // содержит пробел — ошибка
    // int int     = 8;    // совпадает с ключевым словом — ошибка
    // int a-b     = 9;    // содержит дефис — ошибка
    // int $money  = 10;   // символ $ не входит в алфавит C++ — ошибка
}
```

```

cout << myVar << " " << _count << " " << number1
    << " " << totalSum << " " << MAX_VALUE << endl;
return 0;
}

```

Результат выполнения программы:

```
1 2 3 4 5
```

Задание 3. Символы алфавита языка

Таблица символов алфавита языка C++:

Группа символов	Примеры
Буквы	A, b, x, Z, y, _ (подчёркивание)
Цифры	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Специальные символы	+, -, *, /, =, <, >, :, ;, {, }, (,), [,]
Разделители	пробел, табуляция (\t), перевод строки (\n)

Тема 3. Сложные структуры данных: множества

Теоретические вопросы

1. Что такое множество?

Множество — это коллекция уникальных (неповторяющихся) элементов. В C++ реализуется через `std::set` из библиотеки `<set>`. Элементы автоматически упорядочиваются по возрастанию, дубликаты не сохраняются.

2. Какие операции выполняются над множествами?

Основные операции: `insert(x)` — вставить элемент; `erase(x)` — удалить элемент; `find(x)` — найти элемент; `count(x)` — проверить наличие (0 или 1); `size()` — количество элементов; а также объединение, пересечение и разность через алгоритмы STL.

3. Чем множество отличается от массива?

В множестве нет дубликатов, элементы упорядочены автоматически, доступ — через итератор (не по индексу). В массиве допускаются повторения, порядок задаётся вручную, есть прямой доступ по индексу `a[i]`.

4. Что такое пересечение и объединение множеств?

Объединение ($A \cup B$) — все элементы, которые есть хотя бы в одном из множеств. Пересечение ($A \cap B$) — только элементы, которые присутствуют в обоих множествах одновременно. Разность ($A \setminus B$) — элементы, которые есть в A, но отсутствуют в B.

Практическое задание

Задание 1. Создание множества

Создать множество из 10 целых чисел и вывести все элементы и их количество.

```

#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> s = {5, 3, 8, 1, 9, 2, 7, 4, 6, 10};
    cout << "Все элементы множества: ";
    for (int x : s)
        cout << x << " ";
    cout << endl;
    cout << "Количество элементов: " << s.size() << endl;
    return 0;
}

```

Результат выполнения программы:

```

Все элементы множества: 1 2 3 4 5 6 7 8 9 10
Количество элементов: 10

```

Задание 2. Объединение множеств

Даны два множества $A = \{1, 2, 3, 4\}$ и $B = \{3, 4, 5, 6\}$. Найти объединение, пересечение и разность A и B .

```

#include <iostream>
#include <set>
#include <algorithm>
#include <iterator>
using namespace std;

int main()
{
    set<int> A = {1, 2, 3, 4};
    set<int> B = {3, 4, 5, 6};
    set<int> uni, inter, diff;

    set_union(A.begin(), A.end(),
              B.begin(), B.end(), inserter(uni, uni.begin()));

    set_intersection(A.begin(), A.end(),
                    B.begin(), B.end(), inserter(inter,
                    inter.begin()));

    set_difference(A.begin(), A.end(),
                  B.begin(), B.end(), inserter(diff, diff.begin()));

    cout << "Объединение A|B: ";
    for (int x : uni) cout << x << " "; cout << endl;

    cout << "Пересечение A&B: ";
    for (int x : inter) cout << x << " "; cout << endl;

    cout << "Разность A\\B: ";
    for (int x : diff) cout << x << " "; cout << endl;
}

```

```
    return 0;
}
```

Результат выполнения программы:

```
Объединение A|B:  1 2 3 4 5 6
Пересечение A&B:  3 4
Разность A\B:     1 2
```

Задание 3. Практическая задача

Вводятся оценки студентов. Сохранить уникальные оценки в множество, вывести все уникальные значения и их количество.

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> grades;
    int n, grade;
    cout << "Введите количество оценок: ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "Оценка " << i+1 << ": ";
        cin >> grade;
        grades.insert(grade); // дубликаты игнорируются
    }
    cout << "Уникальные оценки: ";
    for (int g : grades)
        cout << g << " ";
    cout << endl;
    cout << "Количество различных оценок: " << grades.size() << endl;
    return 0;
}
```

Результат выполнения программы:

```
Введите количество оценок: 6
Оценка 1: 5
Оценка 2: 4
Оценка 3: 5
Оценка 4: 3
Оценка 5: 4
Оценка 6: 5
Уникальные оценки: 3 4 5
Количество различных оценок: 3
```